Q2 – Loops that Count Attempts

Loop design

A while loop is most natural because blank input must not consume an attempt: increment the counter only after a non-blank value. In a for range(3) loop, continue still consumes an iteration, so it cannot directly satisfy the blank-entry rule without a separate counter-controlled loop. break exits on success; continue skips a blank; pass is a no-operation placeholder and is not needed in finished logic. The loop else runs only if no break occurs.

Changed condition

After a correct security answer, set the attempt limit to five; otherwise retain three. Keep the counter update after blank validation. Avoid updating it in two places, which can cause skipped or unlimited attempts.

Recursion alternative

A recursive verifier calls itself for the next valid attempt. It has O(a) time and O(a) call-stack space for a attempts; loops use O(1) auxiliary space. Recursion adds stack overhead and is unnecessary for a small bounded retry process.

Complexity

Both loops are O(a) time. The iterative implementation is preferred for clarity and constant stack use.

Python Program

```
CORRECT_OTP = "482913"

def verify_while():
    attempts = 0
    while attempts < 3:
        entered = input("OTP: ").strip()
        if not entered:
            print("Blank entry; it does not count.")
            continue
        attempts += 1
        if entered == CORRECT_OTP:
            print("Verified")
            break
        print("Incorrect")
    else:
        print("Account locked for 30 minutes")

def verify_for():
    for attempt in range(3):
        # Inner loop obtains one non-blank attempt; blank entries do not use a turn.
        while True:
            entered = input("OTP: ").strip()
            if entered:
                break
            print("Blank entry; it does not count.")
        if entered == CORRECT_OTP:
            print("Verified")
            break
        print("Incorrect")
    else:
        print("Account locked for 30 minutes")

if __name__ == "__main__":
    verify_while()
```